Chapter 6 (Day 6)

The Classical 5-Stage CPU Pipeline: Fetch, Decode, Execute, Memory, Write-back

6.0 The Single-Cycle Epoch & The RISC Revolution

To appreciate the absolute mechanical brilliance of the modern CPU pipeline, we must examine the physical constraints of early microprocessors. In the late 1970s and early 1980s, processors like the Intel 8086 utilized a **Single-Cycle Architecture** (or un-pipelined multi-cycle architecture). In a strict single-cycle datapath, one instruction is fetched, decoded, executed, memory-accessed, and written back to the register file entirely within a single, continuous clock cycle.

This design created a catastrophic physical bottleneck known as the *Critical Path Dilemma*. The duration of the clock period (T_{clk}) cannot be dynamic; it must be fixed to accommodate the absolute slowest, most complex instruction in the entire Instruction Set Architecture (ISA). If a simple `ADD` takes 20 ns, but a complex `LOAD` from memory takes 100 ns, the motherboard oscillator must be throttled to a 100 ns period (10 MHz). During the `ADD` instruction, the CPU spends 20 ns doing math and 80 ns doing absolutely nothing, bleeding thermal energy while waiting for the clock edge.

The MIPS Project (1981)

In 1981, John Hennessy and his team at Stanford University initiated the **MIPS (Microprocessor without Interlocked Pipelined Stages)** project. Concurrently, David Patterson at UC Berkeley launched the **RISC (Reduced Instruction Set Computer)** project. They observed that 80% of software execution spent its time running only 20% of the available instructions (the 80/20 rule).

Instead of building massively complex, slow processors to handle every edge case, they proposed stripping the CPU down to a minimal, uniform instruction set where every instruction was exactly 32 bits long and did exactly one thing. This uniformity allowed them to slice the physical execution path into five discrete hardware islands separated by D-Flip Flops. By applying Henry Ford's factory assembly-line mechanics directly to silicon, they birthed the **Classical 5-Stage Pipeline**, forever altering the trajectory of Moore's Law and enabling the Gigahertz era.

WHITEBOARD & SYSTEMS WORKSPACE

The RISC vs CISC War: Intel's x86 is a Complex Instruction Set Computer (CISC). Instructions range from 1 byte to 15 bytes. To survive the pipelining revolution, Intel had to build invisible decoders inside the CPU to dynamically translate clunky CISC x86 assembly into elegant, uniform RISC micro-operations (μops) that the internal pipeline could actually digest.

6.1 Latency vs. Throughput: The Core Engineering Distinction

To mathematically analyze pipelining, you must forcefully separate two metrics that high-level software engineers routinely conflate: Latency and Throughput.

- **Latency (Execution Time / Response Time):** The absolute physical time it takes for one specific instruction to clear the processor from the moment it is fetched to the moment its result is written.
- **Throughput (Bandwidth / Execution Rate):** The total volume of instructions completing per unit of time (e.g., Instructions Per Cycle, or IPC).

> **The Pipelining Paradox (Orange)**
> Pipelining a CPU does **not** decrease the latency of an individual instruction. In fact, latency slightly *increases* in a pipelined CPU. Between every stage, hardware engineers must insert Pipeline Registers (D-Flip Flops) to hold the state. These flip-flops introduce setup time (T_{setup}) and clock-to-Q (T_{cq}) propagation delays. Pipelining sacrifices individual instruction latency to massively inflate overall execution throughput.

The Assembly Line Analogy

Consider a car manufacturing plant. It takes 50 hours to build one car from scratch (Latency). In a non-pipelined factory, a team of workers builds one car completely before starting the next. **Throughput: 1 car per 50 hours.**

In a pipelined factory, the process is sliced into 5 distinct stations (Chassis, Engine, Paint, Interior, Testing), each taking 10 hours. The total Latency is still 50 hours per car. However, as soon as Car A leaves the Chassis station and enters the Engine station, Car B enters the Chassis station. Once the pipeline is "primed" (full), a completed car rolls off the Testing line every 10 hours. **Throughput: 1 car per 10 hours (a 5x increase).**

In silicon, if a 5-stage pipeline is perfectly saturated, the CPU retires exactly one instruction every single clock cycle ($IPC = 1.0$), despite each instruction physically taking 5 clock cycles to travel from end to end.

WHITEBOARD & SYSTEMS WORKSPACE

Network Architecture Isomorphism: This exact same latency vs. throughput trade-off applies to network packet routing. Sending a 1GB file over a TCP socket involves slicing it into 1500-byte MTU packets (pipelining). The overall throughput of the network increases via multiplexing, even if the absolute latency of the first byte arriving is identical.

6.2 Stage 1: Instruction Fetch (IF)

The 5-stage classic RISC pipeline breaks execution into five discrete physical hardware islands. The first island is the Instruction Fetch (IF) unit. Its singular goal is to pull raw binary opcodes from memory and feed the beast.

The Program Counter (`RIP` / `PC`)

The CPU maintains a dedicated architectural register pointing to the virtual memory address of the next instruction. In x86-64, this is the `RIP` (Instruction Pointer). In ARM, it is the `PC`.

During the IF stage, the CPU asserts the `RIP` address onto the internal bus connected to the **L1 Instruction Cache (L1i)**. The L1i cache does not return a single instruction; it returns an entire 64-byte Cache Line block. This ensures that sequential instructions are buffered immediately, eliminating future fetch delays.

Instruction Alignment & Pre-Decoding

In a strict RISC architecture (like ARM), every instruction is exactly 4 bytes (32 bits) long. The Fetch unit simply grabs the 4 bytes, increments the `PC` by 4, and sends the data to the Decode stage. It is mechanically trivial.

In an x86-64 CISC architecture, instructions are variable-length. An `inc eax` might be 2 bytes. A `vfmadd231ps` (AVX-512 Fused Multiply-Add) might be 7 bytes. The Fetch unit has no physical way of knowing where one instruction ends and the next begins just by looking at a 64-byte block of RAM. Modern Intel and AMD chips employ heavy **Pre-Decoders** in the IF stage. These circuits scan the byte stream looking for specific instruction prefixes (like the REX prefix or VEX payload) to mark instruction boundaries before passing them to the actual Decoder.

> **The Branch Target Buffer (BTB)**
> If the Fetch unit blindly increments the `RIP` sequentially, it will fetch the wrong code whenever a `for` loop repeats or an `if` statement jumps. The IF stage contains a highly specialized SRAM called the Branch Target Buffer (BTB). It caches the destination addresses of previously executed jumps. If the Fetch unit sees a jump instruction, it queries the BTB to instantly teleport the `RIP` to the correct address without waiting for the ALU to evaluate the jump mathematically.

WHITEBOARD & SYSTEMS WORKSPACE

Code Alignment Hazards: If a hot loop's starting instruction straddles the boundary between two 64-byte cache lines, the Fetch unit must spend two clock cycles pulling both cache lines to assemble the instruction, causing a recurring micro-stall. C++ compilers routinely pad assembly with `NOP` (No Operation) bytes specifically to 16-byte or 64-byte align the targets of jump instructions.

6.3 Stage 2: Instruction Decode & Register Read (ID)

The raw binary bytes retrieved by the Fetch unit are meaningless noise until they are routed through the decoders. The ID stage is the semantic brain of the CPU pipeline.

Micro-operation (μops) Translation

To sustain a 5-stage pipeline, the flow of instructions must be uniform. Because x86 assembly is wildly non-uniform (some instructions touch memory, registers, and the ALU simultaneously), the Decoder acts as a real-time hardware compiler. It intercepts complex x86 instructions and shatters them into tiny, RISC-like operations called micro-operations (μops).

Example: The CISC instruction `add [rbx], eax` (Read memory at RBX, add EAX to it, write the result back to memory at RBX) is impossible to execute in a single pass of a standard 5-stage pipeline. The Decoder violently cracks it into three μops:

1. `LOAD temp_reg, [rbx]`
2. `ADD temp_reg, eax`
3. `STORE [rbx], temp_reg`

The remainder of the pipeline executes these μops, completely oblivious to the original x86 assembly code.

The Register File (RF) & Sign Extension

Once translated, the ID stage must fetch the necessary operands. It sends read requests to the **Register File**, an ultra-fast array of SRAM flip-flops holding the architectural state (RAX, RBX, etc.).

If the instruction includes an immediate constant (e.g., `add rax, -5`), the `-5` is encoded as a small 8-bit integer in the binary stream to save disk space. The ID stage routes this byte through a **Sign-Extension Unit**, which copies the Most Significant Bit (the sign bit, 1) across the remaining 56 bits, converting it into a full 64-bit Two's Complement integer ($0\text{xFFFFFFFFFFFFFFFB}$) before pushing it into the Execute stage. Without sign extension, adding an 8-bit negative to a 64-bit positive would corrupt the mathematics entirely.

WHITEBOARD & SYSTEMS WORKSPACE

The Micro-Op Cache (μop Cache): Decoding complex x86 instructions consumes massive amounts of power and latency. Modern Intel/AMD chips bypass this by storing previously decoded μops in a dedicated L0 Micro-Op Cache. If a loop repeats, the CPU powers down the complex decoders and fetches the pre-digested μops directly, saving watts and cycles.

6.4 Stage 3: Execute (EX)

The Execute stage is where the actual computational work happens. The voltages representing the operands arrive from the ID stage and are routed into highly specialized execution units.

The Execution Ports

A modern CPU is **Superscalar**, meaning it does not have just one ALU. The Execute stage is a massive array of parallel execution ports. A typical core might have:

- **Port 0:** Vector Math (AVX-512 Fused Multiply-Add) and Branch Execution.
- **Port 1:** Standard Integer ALU (Carry-Lookahead Adders, Bitwise XOR/AND).
- **Port 2 & 3:** Address Generation Units (AGU) for Load/Store operations.

Effective Address Generation (AGU)

If the instruction interacts with memory (e.g., `mov rax, [rbp + rdi*8 + 0x10]`), the Execute stage does *not* fetch the data. It calculates the physical pointer. The AGU takes the Base pointer (`RBP`), multiplies the Index (`RDI`) by the Scale (8), adds the Displacement ($0\text{x}10$), and outputs the final 64-bit memory address. This calculation happens in 1 clock cycle using dedicated bit-shifters and adders.

Branch Evaluation & Mispredictions

If the instruction is a conditional jump (`jne` - Jump if Not Equal), the ALU evaluates the condition by checking the Zero Flag (`ZF`) in the `RFLAGS` register. If the Fetch unit (Stage 1) predicted the branch would be Taken, but the Execute unit (Stage 3) mathematically proves it should be Not Taken, a **Branch Misprediction** has occurred.

The EX stage instantly asserts a `FLUSH` signal wire across the entire CPU die. The instructions currently sitting in the Fetch and Decode stages (which were speculatively loaded from the wrong path) are violently aborted, their flip-flops zeroed out. The Fetch unit is redirected to the correct address, and the pipeline sits empty (stalled) for several cycles until the new instructions reach the EX stage.

WHITEBOARD & SYSTEMS WORKSPACE

Branchless Programming: This EX-stage flush penalty is why elite C++ developers use bitwise math (e.g., `val = (x ^ y) & -(x < y);`) instead of `if-else` blocks in hot loops. Bitwise math consumes 3 ALU cycles perfectly deterministically. An `if` statement might consume 1 cycle, or it might trigger a 15-cycle pipeline flush if predicted incorrectly.

6.5 Stage 4: Memory Access (MEM)

The MEM stage is exclusively responsible for interacting with the L1 Data Cache (L1d) and the Memory Management Unit (MMU). It uses the 64-bit pointer calculated by the AGU in the previous stage.

VIPT Caching & The TLB Parallel Lookup

When the CPU accesses memory, it is using a Virtual Address (e.g., `0x00007FF...`). The L1d cache requires a Physical Address in RAM. Translating Virtual to Physical requires querying the Translation Lookaside Buffer (TLB) and potentially walking the OS Page Tables—a massive latency sink.

To do this in exactly 1 clock cycle, modern CPUs use **VIPT (Virtually Indexed, Physically Tagged)** caches. The MEM stage splits the virtual address in half. It uses the lower bits (which are identical in both virtual and physical addresses due to 4KB page alignment) to immediately begin indexing into the L1d SRAM array. Simultaneously, in parallel, it sends the upper bits to the TLB for physical translation. By the time the SRAM array returns the candidate data, the TLB has finished translating the physical tag, which is then compared to verify a cache hit. Parallelization saves the cycle.

Load & Store Buffers

- **Loads:** If the data is in the L1d cache (Hit), it is routed to the next stage. If it is missing (Miss), the pipeline halts, and the Memory Controller reaches out to the L2/L3 cache, causing a severe stall.
- **Stores:** If writing data to memory, the CPU does not wait for the RAM to accept it. It dumps the value into a **Store Buffer** and moves on. The cache coherence protocol flushes the buffer to L1d asynchronously in the background.

> **The Pass-Through Invariant (Emerald)**
> What happens if the instruction is just `add eax, 5`? It does not touch memory at all. In a strict synchronous pipeline, instructions cannot "skip" stages, or they would crash into instructions ahead of them. Non-memory instructions simply **pass through** the MEM stage. The data voltages physically travel through the MEM flip-flops, doing absolutely nothing for one clock cycle, purely to maintain the synchronized marching rhythm.

WHITEBOARD & SYSTEMS WORKSPACE

6.6 Stage 5: Write-Back (WB) & Retirement

The pipeline finishes here. The resulting data—whether mathematically calculated by the ALU in the EX stage, or fetched from the L1d cache in the MEM stage—is physically routed back to the Register File.

Architectural State Commitment

When the clock edge hits in the WB stage, the hardware overwrites the target register (e.g., `RAX`) with the final value. This is the moment the **Architectural State** of the CPU is officially updated. Before this picosecond, the instruction was a "ghost"—a speculative voltage flowing through the silicon.

Precise Exceptions & Interrupt Handling

Why wait until the absolute last stage to write the register? Why not have the ALU write to `RAX` immediately in Stage 3?

Imagine the instruction is `div eax, ebx`, but `EBX = 0`. The ALU triggers a Hardware Divide-by-Zero Exception. If the CPU had eagerly written intermediate results to the registers, the architectural state would be permanently corrupted. The operating system kernel would panic trying to resolve a fractured state.

By strictly delaying all register updates until the WB stage, the CPU achieves **Precise Exceptions**. When the ALU faults in Stage 3, the hardware simply flags the instruction as "poisoned" and lets it flow silently through MEM and WB without writing anything. The architectural state remains perfectly pristine, exactly as it was before the instruction began, allowing the OS kernel to step in, run the exception handler, and potentially resume execution flawlessly.

WHITEBOARD & SYSTEMS WORKSPACE

REF: CH06_WRITEBACK_PRECISE_EXCEPTIONS

Retirement vs. Execution: In modern Out-of-Order CPUs, instructions execute chaotically out of sequence. However, they are collected in a Reorder Buffer (ROB) and forced to Retire (Write-Back) in the exact strict sequence the software developer wrote them. Chaos in execution; perfect order in retirement.

6.7 Pipeline Hazards: Structural & Control

Because five instructions are flowing through the physical hardware simultaneously, physics and logic inevitably collide. These collisions are called **Hazards**. If not mitigated, the CPU must halt the pipeline (injecting "Bubbles" or `NOPs`), destroying the IPC throughput.

1. Structural Hazards

A structural hazard occurs when two instructions attempt to use the exact same physical hardware circuit at the exact same picosecond.

Example: Instruction 1 is in the MEM stage, trying to read a variable from RAM. Instruction 4 is in the IF stage, trying to fetch an opcode from RAM. If the CPU only has one memory bus, they crash.
Resolution: Hardware duplication. This is exactly why the L1 Cache is physically split into L1i (Instruction) and L1d (Data). By providing two distinct physical SRAM arrays, IF and MEM can execute simultaneously without resource contention.

2. Control Hazards

Control hazards are caused by branch instructions (`jmp` , `je` , `call`). The IF stage needs to fetch the next instruction every single clock cycle. But if it encounters an `if` statement, it doesn't know which path to fetch until the EX stage evaluates the condition 2 cycles later.

Resolution: Branch Prediction. The CPU refuses to wait. It uses machine learning algorithms (Branch History Tables and 2-Bit Saturating Counters) to guess whether the branch will be Taken or Not Taken. It speculatively fetches down the guessed path. If the guess is correct (which modern predictors achieve 95%+ of the time), there is zero delay. If the guess is wrong, a pipeline flush occurs, discarding the speculative work.

WHITEBOARD & SYSTEMS WORKSPACE

6.8 Pipeline Hazards: Data Dependencies

Data hazards occur when an instruction depends on the mathematical result of an instruction currently in-flight ahead of it in the pipeline.

Read-After-Write (RAW) / True Dependency

```
Inst 1: add eax, 5       // EX computes eax + 5
Inst 2: sub ebx, eax     // Needs the new 'eax'
```

Let's map this in the 5-stage pipeline. In Cycle 3, Instruction 2 is in the **Decode (ID)** stage trying to read the physical voltage of `EAX` from the Register File. However, Instruction 1 is currently in the **Execute (EX)** stage. It will not write the answer into `EAX` until the **Write-Back (WB)** stage at Cycle 5.

If unmitigated, the hardware must stall Instruction 2 for two complete clock cycles (inserting bubbles) until the WB stage finishes.

The Resolution: Operand Forwarding (Bypassing)

CPU architects cannot afford to stall 2 cycles for every sequential math operation. They solve this using a physical wiring hack called **Operand Forwarding** or Bypassing.

Engineers solder physical copper wires from the output pins of the ALU directly back into the input multiplexers of the ALU. As soon as Instruction 1 computes the answer in the EX stage, the voltage is physically routed backwards across the die. On the very next clock cycle, when Instruction 2 enters the EX stage, the multiplexer ignores the stale value from the Register File and instead consumes the "bypassed" value directly from the ALU output. The RAW hazard is resolved with zero stalled cycles.

> **The Load-Use Stall (Amber)**
>
> Forwarding cannot solve everything. If Instruction 1 is `load eax, [mem]` and Instruction 2 is `add ebx, eax`, the data isn't retrieved from the L1d cache until the MEM stage. The ALU in the EX stage cannot time-travel into the future to get data from the MEM stage. This is a **Load-Use Hazard**. The pipeline MUST physically stall for 1 cycle to allow the MEM stage to forward the data backwards to the EX stage.

6.9 Advanced Hazards & Register Renaming

While classical 5-stage scalar pipelines only suffer from RAW hazards, modern Superscalar Out-of-Order (OoO) CPUs execute multiple instructions chaotically. This introduces False Dependencies.

Write-After-Read (WAR) / Anti-Dependency

```
Inst 1: add ebx, eax     // Needs to read EAX
Inst 2: mov eax, 10      // Writes to EAX
```

If the Out-of-Order engine decides to execute Inst 2 *before* Inst 1 (because Inst 1 is waiting on EBX from a cache miss), it will overwrite EAX. When Inst 1 finally executes, it reads the wrong value (10).

Write-After-Write (WAW) / Output Dependency

```
Inst 1: mov eax, 5
Inst 2: mov eax, 10
```

If Inst 2 executes first, Inst 1 will overwrite the final architectural state with 5, corrupting the program.

The Resolution: Register Renaming (RAT)

WAR and WAW are not true mathematical dependencies; they are artificial bottlenecks caused by the fact that the x86 architecture only has 16 general-purpose registers (RAX, RBX, etc.). The compiler is forced to aggressively reuse them.

Hardware solves this by decoupling the *Architectural Registers* (the 16 registers the software sees) from the *Physical Register File (PRF)*. A modern Intel core actually has over **160 physical registers** hidden in silicon.

The **Register Alias Table (RAT)** intercepts the assembly instructions. When it sees `mov eax, 10`, it dynamically assigns physical register `P45` to act as EAX. When it sees the next `mov eax, 20`, it assigns physical register `P82` to act as EAX. The instructions now write to two completely different hardware locations, instantly eliminating all WAR and WAW hazards and allowing the CPU to execute them simultaneously.

WHITEBOARD & SYSTEMS WORKSPACE

Tomasulo's Algorithm: The dynamic scheduling and register renaming logic used in all modern CPUs is derived from Tomasulo's Algorithm, invented at IBM in 1967 for the System/360 Model 91 floating-point unit.

6.10 Pipeline Mathematics (Speedup & CPI)

As engineers, we must mathematically quantify the exact throughput improvements of pipelining.

- In a single-cycle processor, 1 instruction takes 1 massive cycle. **CPI = 1.0**.
- In an ideal 5-stage pipeline, it takes 5 cycles for the *first* instruction to finish. However, after the pipeline is full, one instruction retires every single clock cycle. The average CPI approaches 1.0, but the clock frequency can be driven 5 times higher because the logic paths are 5 times shorter.

The Speedup Formula

Let k be the number of pipeline stages (e.g., 5). Let N be the number of instructions to execute. Let τ be the clock period (duration of one stage).

Time without pipelining (sequential):
$T_{non\_pipeline} = N \cdot (k \cdot \tau)$

Time with pipelining:
It takes k cycles to push the first instruction out, and $N-1$ cycles for the remaining instructions.
$T_{pipeline} = (k + N - 1) \cdot \tau$

Total Theoretical Speedup (S):
$$S = \frac{T_{non\_pipeline}}{T_{pipeline}} = \frac{N \cdot k}{k + N - 1}$$

> **The Asymptotic Limit (Indigo)**
> As the number of instructions N approaches infinity (a program running for more than a microsecond), the formula simplifies dramatically: $$\lim_{N \to \infty} \frac{N \cdot k}{N} = k$$ The maximum theoretical speedup of a pipeline is exactly equal to the number of stages (k). A 5-stage pipeline makes the CPU 5 times faster.

WHITEBOARD & SYSTEMS WORKSPACE

REF: CH06_PIPELINE_SPEEDUP_MATH

The Pentium 4 Failure: If k stages yield a k-times speedup, why not build a 500-stage pipeline? Intel tried this with the Pentium 4 "Prescott" architecture, building an absurd 31-stage pipeline. The flip-flop latching overhead ate all the performance gains, and branch mispredictions required 31-cycle flushes, destroying overall throughput. Modern architectures (Apple M-series, AMD Zen) stabilized around 14 to 19 stages.

6.11 Software Relevancy: Instruction-Level Parallelism (ILP)

A pipelined CPU relies entirely on **Instruction-Level Parallelism (ILP)**. ILP is the measure of how many operations in a computer program can be performed simultaneously. If code contains strict sequential data dependencies (RAW hazards), the pipeline will violently stall (injecting bubbles), reducing the IPC from an ideal 1.0 down to 0.2.

Code Optimization: Breaking Dependency Chains

Let's look at how C++ and Rust compilers physically rearrange your code to hide pipeline latencies.

```
// Poor ILP (Strict Data Dependency Chain)
int x = data[0];       // Cycle 1: MEM Load
x = x * 5;             // Cycle 2: Wait for Load, then Multiply
x = x + 10;            // Cycle 3: Wait for Multiply, then Add
data[0] = x;           // Cycle 4: Store
```

In the above code, every instruction relies precisely on the output of the previous instruction. The Execute stage sits empty while the Memory stage fetches, and the Memory stage sits empty while the ALU computes. The compiler cannot overlap these instructions.

Compilers use an optimization called **Instruction Scheduling** to interleave independent math between high-latency memory fetches:

```
// High ILP (Interleaved Execution)
int x = data[0];       // MEM stage begins fetch for X
int y = data[1];       // MEM stage begins fetch for Y (Independent)

// The CPU inserts independent math while waiting for memory
int z = independent_counter * 5;

x = x * 5;             // X arrives, EX stage multiplies
y = y * 5;             // EX stage multiplies Y
data[0] = x;
data[1] = y;
```

Loop Unrolling: The most common ILP optimization is Loop Unrolling. Loops introduce control hazards (the `jmp` back to the start). By copy-pasting the loop body multiple times inside the compilation phase, the compiler removes the branches entirely, allowing the CPU to load massive chunks of data sequentially into the pipeline without worrying about branch mispredictions.

6.12 Chapter Verification, Checklist & Quiz

Verification Checklist

I can distinguish Latency (execution time of 1 instruction) from Throughput (instructions retired over time).
I understand why Pipelining increases throughput but actually increases individual instruction latency.
I can name and describe the 5 stages of the classical RISC pipeline (IF, ID, EX, MEM, WB).
I can explain how the Instruction Fetch (IF) stage interacts with the Program Counter (RIP) and the BTB.
I understand the Decode (ID) stage's role in breaking complex x86 assembly into micro-ops ($ \mu\text{ops}$).
I can identify Effective Address Generation (EAG) as a component of the Execute (EX) stage.
I understand the Pass-Through principle for non-memory instructions in the MEM stage.
I can define a Read-After-Write (RAW) data hazard and explain Operand Forwarding (Bypassing).
I understand how Register Renaming and the RAT solve Write-After-Write (WAW) hazards.
I can calculate pipeline Speedup $S = \frac{N \cdot k}{k + N - 1}$ and explain the asymptotic limit k.

WHITEBOARD & SYSTEMS WORKSPACE

REF: CH06_CHECKLIST_NOTES

Comprehensive Examination

1. Which of the following statements perfectly captures the mathematical consequence of converting a single-cycle processor into a 5-stage pipelined processor?

A) The latency of every individual instruction decreases by a factor of 5.
B) The latency of individual instructions increases due to flip-flop latching overhead between stages, but overall instruction throughput increases dramatically.
C) The processor executes 5 distinct software threads concurrently.
D) The CPU avoids branch misprediction penalties by executing all paths simultaneously.
2. During the Execute (EX) stage, an instruction calculates a physical memory address (`[rbp + rcx*4]`). In which pipeline stage is the data actually fetched from the L1 Data Cache?

A) Decode (ID)
B) Execute (EX)
C) Memory Access (MEM)
D) Write-Back (WB)
3. If a 10-stage pipeline executes a massive, infinite loop of instructions, what is the maximum theoretical speedup compared to a non-pipelined processor running the exact same clock period logic?

A) Infinite
B) 5x
C) 10x
D) It depends on the size of the L1 cache.

WHITEBOARD & SYSTEMS WORKSPACE

REF: CH06_QUIZ_SCRATCHPAD_1

4. An instruction sequence is defined as follows: `Instruction 1: mul eax, 5` followed immediately by `Instruction 2: add ebx, eax`. What specific hardware hazard does this create, and how is it mechanically resolved without stalling?

A) Control Hazard; resolved via Branch Prediction.
B) Structural Hazard; resolved by splitting L1i and L1d caches.
C) Read-After-Write (RAW) Data Hazard; resolved by Operand Forwarding (bypassing the register file and routing the ALU output directly back to the ALU input).
D) Write-After-Write (WAW) Hazard; resolved by Register Renaming.

5. Why does the Intel Pentium 4 architecture's massive 31-stage pipeline represent an architectural failure in maximizing throughput?

A) The flip-flop setup times (T_{setup}) exceeded the clock period.
B) The 31 stages generated excessive static leakage power.
C) Branch mispredictions forced the CPU to flush 31 stages of speculative work, obliterating IPC (Instructions Per Cycle) and exposing the massive latency penalties.
D) It required the L1 cache to be unified instead of split.

6. When executing a simple `mov eax, 10` instruction, what physically happens to the data during the Memory Access (MEM) stage of the classical 5-stage pipeline?

A) It writes the value 10 to the L1d cache.
B) It skips the stage entirely and moves to Write-Back.
C) The value `10` passes through the MEM stage flip-flops doing absolutely nothing, maintaining synchronous pipeline rhythm to prevent colliding with other instructions.
D) It updates the Program Counter (`RIP`).

Systems Writing Prompts

Prompt 1: Analyzing ILP and Dependency Chains

Given the following pseudocode, analyze the execution throughput of Loop A vs. Loop B on a 5-stage pipelined processor. Specifically reference the Execute (EX) stage, Data Hazards, and Instruction-Level Parallelism (ILP).

```
// Loop A
for(int i=0; i<N; i++) { sum = sum + A[i]; }

// Loop B
for(int i=0; i<N; i+=2) {
    sum1 = sum1 + A[i];
    sum2 = sum2 + A[i+1];
}
```

Prompt 2: The Stage 5 Architectural Commitment

Explain the critical role of the Write-Back (WB) stage in handling hardware interrupts (like a Divide-by-Zero exception or a Page Fault). Why does delaying the update of the target register (`RAX`) until the absolute final stage protect the CPU's architectural state from corruption?

CHAPTER 6 TECHNICAL ANSWER KEY

1. (B) Pipelining inherently adds intermediate latches (flip-flops) between logic blocks, slightly increasing the absolute latency of a single instruction. However, by overlapping execution, the overall throughput is massively multiplied.

2. (C) The Execute (EX) stage features an Address Generation Unit (AGU) that calculates the 64-bit mathematical pointer location. It does not fetch data. The generated address is passed to the Memory Access (MEM) stage, which actually queries the L1d cache.

3. (C) Using the asymptotic limit formula $\lim_{N \to \infty} \frac{N \cdot k}{k + N - 1}$, as N approaches infinity, the denominator simplifies to N, yielding $N \cdot k / N = k$. A 10-stage pipeline offers a maximum theoretical 10x speedup.

4. (C) Instruction 2 requires `EAX` in the Decode stage, but Instruction 1 hasn't written `EAX` to the register file yet. This RAW hazard is solved via Operand Forwarding, which wires the output of the EX stage ALU directly back into the input multiplexer of the EX stage for the very next cycle.

5. (C) Deep pipelines rely on keeping the pipeline full. A branch instruction forces the CPU to guess the path. When a 31-stage pipeline guesses wrong, it has fetched 30 invalid instructions that must be mathematically aborted. The CPU stalls, obliterating the theoretical IPC gains.

6. (C) To maintain the strict synchronous rhythm of the pipeline, instructions cannot jump stages. Non-memory instructions simply pass their data through the MEM flip-flops harmlessly, arriving at the WB stage exactly on schedule.

Prompt 1: Loop A forces a strict Read-After-Write (RAW) data dependency chain. The ALU cannot compute the next `sum` until the previous iteration writes the result. The pipeline stalls. Loop B computes `sum1` and `sum2` independently. The Execute (EX) stage can interleave these calculations perfectly (or execute them concurrently in a superscalar architecture), saturating Instruction-Level Parallelism (ILP) and nearly doubling the throughput IPC.

Prompt 2: If an instruction triggers a fault (e.g., dividing by zero in the EX stage), the CPU must abort the instruction. If the CPU had eagerly written intermediate results to the Register File during EX or MEM, the architectural state would be permanently corrupted. By strictly delaying register updates until the WB stage, the CPU can simply drop the instruction mid-pipeline, ensuring the software environment remains perfectly deterministic and uncorrupted (Precise Exceptions).

WHITEBOARD & SYNTHESIS WORKSPACE

REF: CH06_FINAL_SYNTHESIS